



Telecom Usage & Profile Enrichment Analytics — LLD (Q402)

1) Problem Statement

You are given a telecom events dataset (CSV) that tracks user usage and network quality (data, voice, sms, roaming, latency, dropped calls, device, etc.).

You must implement **pure PySpark functions** to:

- Define schema and load telecom usage events
- Parse event dates
- Create derived KPI columns (total activity, usage score, network risk flags)
- Filter and aggregate usage / network metrics by different dimensions (city, plan, device, date)
- Return top-N analytics outputs
- Additionally, create and join a **small inline dataset** (profiles + plans) to practice **join + null enrichment**
- Implement two functions using **unix_timestamp** conversions:
 - convert event date to epoch seconds
 - convert epoch seconds back to timestamp

✓ All operations must use Spark transformations (no Python loops for data logic).

2) Input Datasets

A) Main dataset (CSV)

Path: `data/telecom_usage.csv`

Expected columns (loaded as strings/numbers per schema):

- `event_id` (string)
- `user_id` (string or int depending on schema; must be consistent)
- `plan` (string)
- `city` (string)
- `event_date` (string in `yyyy-MM-dd`)
- `data_mb` (double)
- `voice_minutes` (double/int)
- `sms_count` (int)
- `roaming` (string flag "Y" / "N")
- `device_type` (string)
- `dropped_calls` (int)
- `latency_ms` (int)

B) Inline join dataset (created inside Spark via createDataFrame)

No CSV files for this piece — must be created inline.

Profiles dataset (inline)

Columns:

- `user_id` (int)
- `first_name` (string)
- `last_name` (string)
- `full_name` (string, may be null)
- `plan_id` (string)

```
profiles_rows = [
```

```

# user_id, first_name, last_name, full_name,    plan_id
(201, "Aarav", "Iyer", None, "P1"),

(202, "Diya", "Sharma", "Diya Sharma", "P2"), # already filled → must remain
unchanged

(203, "Kabir", "Mehta", None, "P2"),

(204, "Meera", "Nair", None, "P3"),

(205, "Rohan", "Singh", "Rohan S.", "P3"), # prefilled (non-standard) must remain
unchanged

(206, "Sara", "Khan", None, "P99"), # plan_id not present → join behavior
check

]

```

Plans dataset (inline)

Columns:

- `plan_id` (string)
- `plan_name` (string)

```

plans_rows = [

# plan_id, plan_name

("P1", "Basic"),

("P2", "Plus"),

("P3", "Premium"),

]

```

3) Required Imports (Recommended)

```
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql.types import StructType, StructField, StringType,
IntegerType, DoubleType
from pyspark.sql import functions as F
```

4) Functions to Implement (27 total)

A) Core Telecom Functions (original set)

1. `define_schema()` -> `StructType`

Defines explicit schema for `telecom_usage.csv` with correct types for all fields (nullable).

2. `load_data(spark: SparkSession, path: str, schema: StructType)` -> `DataFrame`

Loads telecom CSV using provided schema and header.

3. `parse_event_date(df: DataFrame)` -> `DataFrame`

Converts `event_date` from string to `DateType` using `to_date`.

4. `add_total_activity(df: DataFrame)` -> `DataFrame`

Adds:

- `total_activity = voice_minutes + sms_count`
-

5. `add_usage_score(df: DataFrame)` -> `DataFrame`

Adds a usage score column.

Business rule (recommended):

- `usage_score = data_mb + (voice_minutes * 0.5) + (sms_count * 0.1)`
(Exact weights may be enforced by tests; follow LLD if provided in your assessment notes.)
-

6. `filter_roaming_events(df: DataFrame) -> DataFrame`

Returns only roaming events.

Rule:

- roaming flag is "Y" / "N" and "Y" is allowed.
-

7. `top_n_users_by_data(df: DataFrame, n: int) -> DataFrame`

Groups by `user_id`, sums `data_mb` as `total_data_mb`, orders desc by total, tie-break by `user_id` asc, returns top `n`.

8. `avg_latency_by_city(df: DataFrame) -> DataFrame`

Groups by `city`, computes average `latency_ms` as `avg_latency_ms`.

9. `most_used_plan(df: DataFrame) -> str`

Returns the plan name/value that appears most frequently (by count).

Tie-break recommendation:

- plan ascending.
-

10. `dropped_call_rate_by_plan(df: DataFrame) -> DataFrame`

For each plan compute:

- `total_calls = count(*)`
- `dropped_total = sum(dropped_calls)`
- `dropped_call_rate = dropped_total / total_calls`

Return columns:

- `plan, dropped_call_rate`
-

11. `high_latency_events(df: DataFrame, threshold: int) -> DataFrame`

Filters rows where `latency_ms > threshold`.

12. `count_users_per_city(df: DataFrame) -> DataFrame`

Counts distinct users per city.

Return columns:

- `city, user_count`
-

13. `daily_data_trend(df: DataFrame) -> DataFrame`

Groups by `event_date`, sums `data_mb` as `daily_data_mb`, orders by date asc.

14. `top_device_by_usage_score(df: DataFrame) -> str`

Compute average or total `usage_score` per `device_type` (per LLD/test), return top device type by that metric.

15. `list_cities(df: DataFrame) -> list_str`

Return sorted list of unique cities:

- `.select("city").distinct()` then collect to Python list.
-

16. `events_in_date_range(df: DataFrame, start: str, end: str) -> DataFrame`

Filters where `event_date` is between start and end inclusive.

Assumption:

- `event_date` already parsed to `DateType`.
-

17. `flag_network_risk(df: DataFrame, latency_threshold: int, drop_threshold: int) -> DataFrame`

Adds boolean (or string) risk flag column.

Recommended logic:

- `is_risky = (latency_ms > latency_threshold) OR (dropped_calls > drop_threshold)`
-

18. `top_n_risky_users(df: DataFrame, n: int) -> DataFrame`

From risky rows only (use `is_risky == True`), group by `user_id` and count risky events, order desc by risky count, tie-break `user_id` asc, return top n.

19. `avg_data_per_plan(df: DataFrame) -> DataFrame`

Group by `plan`, compute `avg(data_mb)`.

20. `get_heaviest_user(df: DataFrame) -> tuple`

Returns:

- `(user_id, total_data_mb)` where `total_data_mb` is the sum of `data_mb` for that user.

Tie-break:

- smallest `user_id` if totals tie.
-

B) Added Join + Enrichment + Unix Timestamp Functions (new set)

21. `define_profile_schema() -> StructType`

Defines schema for inline profiles dataset:

- `user_id` (int)
 - `first_name` (string)
 - `last_name` (string)
 - `full_name` (string, nullable)
 - `plan_id` (string)
-

22. `define_plan_schema() -> StructType`

Defines schema for inline plans dataset:

- `plan_id` (string)
 - `plan_name` (string)
-

23. `load_inline_profile_data(spark: SparkSession, profile_schema: StructType, plan_schema: StructType) -> tuple_df`

Creates two DataFrames using **inline tuples + schemas**:

- `profiles_df`
- `plans_df`

✓ Must use:

- `spark.createDataFrame(inline_rows, schema=profile_schema)`
- `spark.createDataFrame(inline_rows, schema=plan_schema)`

Returns:

- `(profiles_df, plans_df)`
-

24. `join_profile_plan(profiles_df: DataFrame, plans_df: DataFrame) -> DataFrame`

Joins profiles with plan lookup on `plan_id`.

Output should include (recommended):

- `user_id, first_name, last_name, full_name, plan_id, plan_name`
-

25. `enrich_full_name(profile_joined_df: DataFrame) -> DataFrame`

Enriches missing `full_name`.

Rule:

- If `full_name` is null → `concat_ws(" ", first_name, last_name)`

- If not null → keep as-is

Must ensure:

- no null values remain in `full_name` after enrichment
-

26. `add_event_epoch_seconds(df: DataFrame) -> DataFrame`

Adds an epoch column using `unix_timestamp`.

Rule:

- `event_epoch_seconds = unix_timestamp(event_date)`
(If `event_date` is `DateType`, cast/format as needed.)

Output:

- original columns + `event_epoch_seconds` (long/int)
-

27. `add_event_ts_from_epoch(epoch_df: DataFrame) -> DataFrame`

Takes a `DataFrame` that contains `event_epoch_seconds` and converts it back to timestamp/datetime column.

Rule:

- `event_ts = from_unixtime(event_epoch_seconds).cast("timestamp")`

Output:

- original columns + `event_ts`
-

5) Constraints & Rules (Trainee Guidance)

✓ Allowed Spark APIs:

- `withColumn`, `filter/where`, `groupBy`, `agg`, `orderBy/sort`, `join`, `when`, `concat_ws`, `coalesce`, `unix_timestamp`, `from_unixtime`, `to_date`

✗ Not allowed:

- Hardcoding real dataset outputs
- Python loops / list logic to replace Spark transformations
- `spark.createDataFrame()` anywhere **except** `load_inline_profile_data`

✓ 'Y' / 'N' flags are allowed and are not considered hardcoding.

6) Return Types (as per test_config.json)

Return type may be one of:

- `DataFrame`
- `tuple_df`
- `list_str`
- `tuple`
- `int`, `float`, `str`
- `StructType`